

УНИВЕРЗИТЕТ У БЕОГРАДУ
ЕЛЕКТРОТЕХНИЧКИ ФАКУЛТЕТ



**ПОБОЉШАЊЕ ТАЧНОСТИ НУМЕРИЧКИХ
РЕЗУЛТАТА У ПАРАЛЕЛНОМ ОКРУЖЕЊУ ПРИМЕНОМ
КАХАНОВОГ И НОЈМАЈЕРОВОГ АЛГОРИТМА**

Дипломски рад

Ментор:

проф. др Марко Мишић

Кандидат:

Урош Недељковић 2022/0058

Београд, септембар 2026.

САДРЖАЈ

САДРЖАЈ	2
1. УВОД.....	3
2. ПРЕДСТАВЉАЊЕ БРОЈЕВА У ПОКРЕТНОМ ЗАРЕЗУ	5
2.1. IEEE-754 СТАНДАРД	5
2.1.1. Формат.....	5
2.1.2. Заокруживање.....	7
2.1.3. Изузетне ситуације.....	8
2.2. ГРЕШКЕ ПРИЛИКОМ ЗАОКРУЖИВАЊА	9
2.3. ПРОБЛЕМ НЕДОСТАТКА АСОЦИЈАТИВНОСТИ ПРИЛИКОМ САБИРАЊА БРОЈЕВА У ПОКРЕТНОМ ЗАРЕЗУ	10
3. НУМЕРИЧКА СТАБИЛНОСТ У ПАРАЛЕЛНОМ ОКРУЖЕЊУ	12
3.1. О ПРОБЛЕМУ НУМЕРИЧКЕ СТАБИЛНОСТИ У ПАРАЛЕЛНОМ ПРОГРАМИРАЊУ	12
3.2. АЛГОРИТМИ ЗА СТАБИЛНО СУМИРАЊЕ	12
3.2.1. Каханов алгоритам.....	13
3.2.2. Нојмајеров алгоритам.....	15
4. ПРЕГЛЕД КОРИШЋЕНЕ ТЕХНОЛОГИЈЕ И ТЕСТ ОКРУЖЕЊА	17
4.1. OPENMP	17
4.2. ТЕСТ ПЛАТФОРМА	18
4.3. ТЕСТ АПЛИКАЦИЈЕ.....	18
4.3.1. Сумирање низа (<i>sumArray</i>).....	19
4.3.2. Множење матрица (<i>matMul</i>).....	19
4.3.3. Молекуларна динамика (<i>md</i>)	19
5. ОПИС ИМПЛЕМЕНТАЦИЈЕ	20
5.1. ЗАЈЕДНИЧКИ АСПЕКТИ ИМПЛЕМЕНТАЦИЈЕ ТЕСТ АПЛИКАЦИЈА	20
5.2. СПЕЦИФИЧНОСТИ ИМПЛЕМЕНТАЦИЈЕ ТЕСТ АПЛИКАЦИЈА	22
5.2.1. Специфичности множења матрица.....	22
5.2.2. Специфичности молекуларне динамике.....	23
6. РЕЗУЛТАТИ И ДИСКУСИЈА.....	25
6.1. РЕЗУЛТАТИ ПРОГРАМА ЗА СУМИРАЊЕ НИЗА	25
6.2. РЕЗУЛТАТИ ПРОГРАМА ЗА МНОЖЕЊЕ МАТРИЦА	28
6.3. РЕЗУЛТАТИ ПРОГРАМА МОЛЕКУЛАРНА ДИНАМИКА	31
7. ЗАКЉУЧАК	37
8. ЛИТЕРАТУРА	39
СПИСАК СКРАЋЕНИЦА	40
СПИСАК СЛИКА	41
СПИСАК ТАБЕЛА	42

1. УВОД

Аритметика бројева у покретном зарезу представља стандардан начин представљања реалних бројева у рачунарским системима. Ипак, услед тога да је могуће представити само коначан подскуп реалних бројева, свака аритметичка операција може унети грешку заокруживања, па се у случају сабирања великог броја вредности може акумулирати и значајно нарушити тачност коначног резултата. Услед ове непрецизности, сабирање бројева у покретном зарезу не поседује својство асоцијативности. Овај проблем је директно повезан са паралелним програмирањем, чији редослед комбиновања није фиксиран, различита груписања сабирака могу дати различите резултате.

Један од начина да се ублажи проблем акумулације грешке заокруживања јесте примена алгоритама за стабилно сумирање, међу којима су најпознатији Каханов и Нојмајеров алгоритам. Ови алгоритми прате грешку насталу у сваком кораку сабирања и накнадно је компензују, чиме се постиже већа тачност у односу на наивно сабирање.

Циљ овог рада јесте да се испита на који начин се Каханов и Нојмајеров алгоритам могу применити у паралелном OpenMP окружењу, као и колики утицај њихова примена има на тачност и брзину извршавања у односу на наивно сабирање. Због тога су алгоритми имплементирани и тестирани на три апликације различите сложености: сумирању низа, множењу матрица и симулацији молекуларне динамике, при чему су резултати упоређивани и у секвенцијалном и у паралелном окружењу.

У другом поглављу дат је преглед представљања бројева у покретном зарезу према IEEE-754 стандарду, са освртом на изворе грешака заокруживања и последице недостатка асоцијативности приликом сабирања. Треће поглавље бави се проблемом нумеричке стабилности у паралелном окружењу, уз детаљан приказ Кахановог и Нојмајеровог алгоритма. У четвртном поглављу су представљени технологија OpenMP, тест платформа и тест апликације. Пето поглавље садржи опис имплементације Кахановог и Нојмајеровог алгоритма и кориснички дефинисаних редукција за ова два типа сумирања, као и начин рачунања референтне вредности високе тачности. У шестом поглављу приказани су и

анализирани добијени резултати, кроз поређење секвенцијалних и паралелних верзија са различитим бројем нити наивног, Кахановог и Нојмајеровог сумирања. Седмо поглавље садржи закључак и предлоге за унапређење рада.

2. ПРЕДСТАВЉАЊЕ БРОЈЕВА У ПОКРЕТНОМ ЗАРЕЗУ

У овом поглављу биће представљени бројеви са покретним зарезом и аритметичке операције над њима, који су важни за разумевање нумеричких грешака у рачунарству. Прво ће бити описан IEEE-754 стандард, који дефинише бинарне и децималне формате и правила аритметике бројева у покретном зарезу. Затим следи приказ проблема који нарушавају тачност резултата - грешке приликом заокруживања и одсуство асоцијативности.

2.1. IEEE-754 стандард

IEEE-754 је стандард који дефинише начин представљања бројева у покретном зарезу у рачунарском окружењу униформно, независно од хардвера на ком се извршава. То омогућава да се на конзистентан начин приказују реални бројеви. Међутим, с обзиром на то да се представља само њихов коначни подскуп, појављују се грешке у запису или операцијама са реалним бројевима.

2.1.1. Формат

Стандард прописује пет основних формата: три бинарна (зависно од дужине записа: *binary32*, *binary64* и *binary128*) и два децимална (*decimal64* и *decimal128*). Њиме су дефинисани и додатни формати који повећавају прецизност, али за потребе овог рада нису од претераног интереса.

Већи број битова у запису броја повећава прецизност. Тако формат *binary32* представља једноструку тачност (тип *float* у већини програмских језика), док *binary64* означава двоструку тачност (*double*), који ће бити главни фокус у овом раду.

Бројеви у покретном зарезу у овом стандарду се представљају помоћу следећих целобројних параметара:

- b = основа (radix), 2 или 10
- p = број цифара у мантиси (прецизност)
- e_{max} = максимална вредност експонента

- e_{min} = минимална вредност експонента (дефинисана као $1-e_{max}$).

Табела 2.1.1.1. Параметри основних формата бројева у покретном зарезу

	Бинарни формат ($b=2$)			Децимални формат ($b=10$)	
параметар	<i>binary32</i>	<i>binary64</i>	<i>binary128</i>	<i>decimal64</i>	<i>decimal128</i>
p	24	53	113	16	34
e_{max}	127	1023	16383	384	6144

У табели 2.1.1.1. дати су параметри број цифара у мантици и максимална вредност експонента у зависности од основе и дужине записа броја.

Сваки број се у бинарном формату ($b=2$) може представити помоћу три поља:

- 1) Поље знака (*sign*, S) заузима бит који одређује знак броја. Вредност $S=0$ означава позитиван, док $S=1$ негативан број.
- 2) Поље експонента (*exponent*, E) означава ред величине броја. Стварни експонент се рачуна помоћу формуле $e = E - bias$, па његова вредност може бити и негативна. Поље експонента је на овај начин увек ненегативна вредност, па је тако хардверски погодније за представљање. Заузима ширину $w = 8$ битова за *binary32*, што је 256 различитих образаца. Ипак, само вредности $E = 1, \dots, 254$ кодирају нормалне бројеве, са стварним експонентима у опсегу од e_{min} до e_{max} (-126, ..., 127), док су вредности $E = 0$ и $E = 255$ резервисане за посебне случајеве. Аналогно, поље експонента за формат *binary64* заузима ширину $w = 11$ битова, што је укупно 2048 могућих образаца, од којих се за нормалне бројеве користе вредности $E = 1, \dots, 2046$, а резервисане за посебне случајеве су $E = 0$ и $E = 2047$.
- 3) Поље мантисе (*significand*, F)¹ представља бинарни запис разломљеног дела мантице, без водеће цифре. За нормалне бројеве, водећа цифра мантисе увек износи 1 и не чува се експлицитно у меморији (скривени бит), већ се подразумева приликом рачунања броја, док за субнормалне бројеве водећа цифра износи 0. За ово поље преостају 23 бита за *binary32* односно 52 за *binary64* формат, што је за један бит мање од укупног броја цифара мантисе, пошто се водећа цифра не мора чувати.

¹ За потребе овог рада усвојена је једноставнија нотација, уместо $2^{1-p} \times T$ из литературе користи се F , јер је то довољно за разумевање основних концепата, а прегледније је за праћење.

Нормалан коначан број представља се формулом $x = (-1)^S \times 2^{E-bias} \times (1 + F)$, при чему је $0 \leq F < 1$ и $bias$ има вредност 127 за *binary32*, односно 1023 за *binary64* формат, што одговара вредностима e_{max} у оба формата. На овај начин се постиже баланс у представљању малих и великих бројева, односно стваран експонент је у одговарајућем опсегу од e_{min} до e_{max} .

Према IEEE-754 стандарду, на основу вредности поља експонента (E) и поља мантисе (F), разликује се пет случајева (при чему је w ширина поља експонента):

- 1) Ако је $E = 2^w - 1$ и $F \neq 0$, односно кад су сви битови експонента јединице и мантиса различита од нула, тада број има вредност NaN (*Not-a-Number*), који зависно од вредности мантисе може бити:
 - i. qNaN (*Quiet NaN*), који не сигнализира изузетак, већ аритметичке операције у којима учествује враћају вредност qNaN као резултат,
 - ii. sNaN (*Signaling NaN*), који приликом употребе у аритметичкој операцији сигнализира изузетак постављањем статусне заставице.
- 2) Ако је $E = 2^w - 1$ (сви битови експонента су јединице) и $F = 0$, онда је вредност у бесконачности, а знак се рачуна на основу бита S по формули $(-1)^S \times \infty$.
- 3) Ако је $1 \leq E \leq 2^w - 2$, онда је представљен нормалан број, чија се вредност рачуна по формули $(-1)^S \times 2^{E-bias} \times (1 + F)$.
- 4) Ако је $E = 0$ и $F \neq 0$, онда је представљен субнормалан број, који има имплицитну водећу цифру 0 уместо 1. Тада се његова вредност рачуна по формули $(-1)^S \times 2^{e_{min}} \times (0 + F)$.
- 5) Ако је $E = 0$ и $F = 0$, онда је представљен број нула, с тим да постоје две репрезентације зависно од S . То су позитивна и негативна нула. [1]

2.1.2. Заокруживање

IEEE-754 стандард дефинише заокруживање као поступак којим се резултат аритметичке операције, након што је израчунат са бесконачном прецизношћу и неограниченим опсегом, прилагођава формату резултата. Дефинисано је више начина заокруживања, који се називају атрибути правца заокруживања и подељени су у две различите групе:

- 1) **закруживање на најближу вредност** – Закружује се на најближи број који је унутар подскупа вредности реалних бројева који се могу представити у датом формату. Ако је апсолутна вредност коју треба заокружити већа или једнака $b^{e_{max}} \times \left(b - \frac{1}{2}b^{1-p}\right)$, онда се заокружује на $(-1)^S \times \infty$. Постоје две подгрупе у случају да је вредност тачно на средини између два суседна броја:
 - i. *roundTiesToEven* је подразумевани начин заокруживања за бинарни формат. У случају једнаке удаљености бира се кандидат са парном последњом цифром значајног дела. Овај начин је пожељан, поготово за дуже прорачуне, јер смањује системску пристрасност у односу на увек истосмерно разрешавање изједначења.
 - ii. *roundTiesToAway* бира најближи представљив број, а само када су два кандидата једнако удаљена бира онај веће апсолутне вредности.
- 2) **усмерено заокруживање** – Постоје три подгрупе:
 - i. *roundTowardPositive* – Бира се број у покретном зарезу датог формата (по потреби $+\infty$) који је најближи бесконачно прецизном резултату, али није мањи од њега.
 - ii. *roundTowardNegative* – Бира се број (по потреби $-\infty$) који је најближи бесконачно прецизном резултату, али није већи од њега.
 - iii. *roundTowardZero* – Бира се број који је најближи бесконачно прецизном резултату, али није већи по апсолутној вредности од њега, заокружује се ка нули.

2.1.3. Изузетне ситуације

Појавом изузетка поставља се одговарајућа статусна заставица, која омогућава детекцију губитка тачности и других нумеричких проблема. Они не морају узроковати прекид, па се на тај начин олакшава анализа нумеричке стабилности. Стандард дефинише пет врста изузетака који се могу сигнализирати током операције над бројевима у покретном зарезу:

- 1) Неважећа операција (*Invalid operation*) се поставља као заставица приликом извршавања операција као што су: $\frac{0}{0}$, $\frac{\infty}{\infty}$, $\infty - \infty$, $0 \times \infty$.

- 2) Делење нулом је изузетак који се сигнализира када се коначна вредност различита од нуле дели нулом. Пошто и нула поседује знак, у зависности од бита S оба операнда одређује се знак бесконачности.
- 3) Прекорачење (*Overflow*) настаје када се достигне вредност која је по апсолутној вредности већа од највеће коначне вредности и тада се поставља одговарајућа заставица. Зависно од начина заокруживања описаних у поглављу 2.1.2, резултат се заокружује на одговарајућу вредност. Паралелно извршавање програма може довести до прекорачења које се при другом редоследу не јавља, или спречити прекорачење до којег би иначе дошло.
- 4) повећава шансу за појављивањем овог изузетка. Услед промене распореда операција, резултат у једном случају може довести до прекорачења и бесконачности, док се при другачијем редоследу то не би догодило.
- 5) Поткорачење (*Underflow*) се детектује када је резултат операције изузетно мала ненулта вредност, која је мања по апсолутној вредности од најмањег нормалног броја у датом формату. У том случају број се представља као субнормалан или нула. Уколико долази до губитка прецизности, поставља се заставица поткорачења и сигнализира изузетак непрецизног резултата.
- 6) Непрецизан резултат (*Inexact*) се сигнализира у случају да се заокружени резултат разликује од резултата који би био добијен када би опсег експонента и прецизности били неограничени, осим ако није другачије дефинисано. Заставица се може поставити самостално, ако операција не изазива ниједан други изузетак, а подразумевано се увек поставља уз *Overflow* или *Underflow* заставицу.

2.2. Грешке приликом заокруживања

Реални бројеви на рачунару се могу приказати искључиво помоћу коначног броја вредности, коришћењем апроксимације одговарајућим подскупом. Разлог томе је чињеница да рачунар за представљање сваког броја користи фиксиран, коначан број битова, чиме је ограничен и укупан број различитих образаца који се могу записати, па тако за формат од n битова, могуће је записати тачно 2^n различитих образаца. Пошто скуп реалних бројева, чак и унутар произвољно малог интервала садржи бесконачно много елемената, а рачунар располаже са коначним бројем битских образаца морају постојати реални бројеви који

немају свој тачан битски запис, Последица тога је да се такви бројеви морају заокружити на најближи број који је могуће представити у датом формату, чиме скуп представљених бројева постаје дискретан подскуп читавог скупа реалних бројева.

ULP (*Unit in the last place*) се дефинише као разлика између две суседне вредности које се могу представити датим форматом бројева са покретним зарезом. С обзиром да зависи од вредности експонента *ulp* није константан кроз читав опсег представљања бројева. За подразумевани начин заокруживања (*roundTiesToEven*), апсолутна грешка приликом заокруживања за појединачну вредност највише износи половину вредности *ulp*, док код усмерених заокруживања грешка (*roundTowardPositive*, *roundTowardPositive*, *roundTowardZero*) може максимално износити целу вредност *ulp*.

Поништавање (*Cancellation*) је појава која настаје када се одузимају два броја која су приближно једнака, па добијена разлика често има велику релативну грешку. Ипак, у модерним рачунарима се хардверски често користе заштитне цифре (*Guard Digits*) које омогућавају да се ова грешка смањи. Постоје две врсте:

- 1) Катастрофално поништавање (*Catastrophic cancellation*) настаје када операнди већ садрже грешке заокруживања, настале у претходним корацима рачунања. На пример, при решавању квадратне једначине и коришћењу формуле $b^2 - 4ac$, вредности b^2 и $4ac$ су већ заокружене, па ће често највећи део грешке бити узрокован заокруживањем тих операција, а не самог одузимања.
- 2) Бенигно поништавање (*Benign cancellation*) настаје када су вредности које се одузимају веома прецизне, односно када су њихове претходне грешке контролисане, па одузимање блиских представљених бројева може дати тачан резултат или веома прецизан резултат. [2]

2.3. Проблем недостатка асоцијативности приликом сабирања бројева у покретном зарезу

Услед грешака у заокруживању међурезултата при извршавању више узастопних операција сабирања присутан је недостатак асоцијативности. Такође, као што је раније напоменуто, редослед сабирака може одредити да ли се сигнализира изузетак прекорачења. На пример, ако би се прво сабирале само позитивне вредности, па затим негативне, међузбир би могао премашити максималну вредност и бити представљен као бесконачност. На тај начин негативни елементи не би утицали на коначан резултат, јер би те операције биле без

ефекта. Ако би позитивни и негативни сабирци били распоређени наизменично, укупна вредност збира евентуално не би премашила максималну дозвољену границу.

Још једна ситуација у којој често настаје грешка приликом заокруживања је када се сабирају вредности које нису истог реда величине. У таквим случајевима, мања вредност може бити у потпуности занемарена. То је један од примера који може нарушити асоцијативност. За илустрацију овог примера нека се за променљиве A , B и C иницијализују следеће вредности:

```
double A = 1.0;  
double B = 1.0 * 1e16;  
double C = -1.0 * 1e16;
```

Код 2.3.1. Иницијализација променљивих

Тачна вредност збира $A+B+C$ износи 1.0 . Ипак, уколико се на рачунару тим редоследом врши сабирање, резултат ће бити 0.0 . Разлог је заокруживање међурезултата сабирања $A+B$ на B , односно потпуно занемаривање вредности променљиве A . Ако би се променио редослед на $(A+C)+B$ добио би се исти исход, будући да би A било занемарено при сабирању са C . Док би за збир $(B+C)+A$ вредност била 1.0 , што је и математички тачна вредност сабирања. Тај исход се постиже зато што међурезултат сабирања $B+C$ износи нула, која је блиска вредности A , па не долази до занемаривања вредности A услед велике разлике у експонентима нити до појаве грешке приликом заокруживања.

Операција сабирања за бројеве у покретном зарезу је комутативна, односно за два сабирка израчунава исти збир независно од њиховог редоследа ($a+b=b+a$).

3. НУМЕРИЧКА СТАБИЛНОСТ У ПАРАЛЕЛНОМ ОКРУЖЕЊУ

У овом поглављу биће објашњено зашто настаје проблем нумеричке стабилности и приказују се начини решавања коришћењем алгоритама за стабилно сумирање. Посебна пажња ће бити стављена на Каханов и Нојмајеров алгоритам, који повећавају тачност резултата при сумирању бројева у покретном зарезу, праћењем грешке приликом заокруживања.

3.1. О проблему нумеричке стабилности у паралелном програмирању

Као што је напоменуто у потпоглављу 2.3, својство асоцијативности при сабирању бројева у покретном зарезу није задовољено. Односно редослед извршавања операција утиче на коначан збир.

Овај проблем је посебно присутан у паралелним програмима, где редослед операција није унапред одређен. Свака нит (процес) израчунава свој део резултата независно од других, па није познато којим редом ће нити завршавати своје извршавање. Због тога се током операције редукције мења распоред сабирака. Самим тим се за исти програм и исте улазне параметре може добити различит коначан резултат, што указује на присуство недетерминизма.

На конкретном једноставном примеру са три нити, где су доприноси нити пре редукције 1.0, 10^{16} и -10^{16} може се показати проблем недетерминизма. Како OpenMP не прописује редослед комбиновања, а пошто је показано у потпоглављу 2.3. да збир ова три сабирака у покретном зарезу зависи од њиховог редоследа, резултујућа вредност није детерминистичка, односно може, али не мора да се разликује у поновљеним покретањима.[6]

3.2. Алгоритми за стабилно сумирање

Један од приступа који може ублажити претходно наведени проблем је примена алгоритама за стабилно сумирање. Ови алгоритми не обезбеђују својство асоцијативности при сабирању бројева у покретном зарезу, али значајно смањују грешку заокруживања. На

тај начин се постиже боља нумеричка стабилност извршавања програма, односно већа тачност коначног резултата.

Основна идеја алгоритама за стабилно сумирање је праћење грешке заокруживања, која настаје у сваком кораку, са циљем да се што је могуће више компензује у наредним операцијама. Као представници ове групе, у наставку биће детаљно приказани Каханов и Нојмајеров алгоритам.

3.2.1. *Каханов алгоритам*

У оквиру овог одељка биће представљена прва верзија Кахановог алгоритма, која има велики историјски значај и у оквиру које је представљена идеја самог алгоритма, затим модерна верзија, која ће бити коришћена у самој имплементацији. На крају ће бити појашњено понашање Кахановог алгоритма у паралелном окружењу на основу података из литературе.

i) Прва верзија Кахановог алгоритма

Технику исправљања грешке приликом сабирања коришћењем акумулације Кахан први пут представља 1965. године. [3] Тада је приликом извођења великог броја аритметичких операција бројева у покретном зарезу, обично писаних у језику FORTRAN, постојала значајна грешка у заокруживању. Пре него што је аритметика дупле тачности постала стандардна и широко доступна, како би побољшао тачност резултата, Кахан је примењивао следећи алгоритам:

```
S = 0.0
S2 = 0.0
DO 4 I = 1, N
  YI = ...
  S2 = S2 + YI
  T = S + S2
  S2 = (S-T) + S2
  S = T
  ...
```

Код 3.2.1.1. Каханов алгоритам – изворни FORTRAN код за компензовано сабирање [3]

На почетку се иницијализују сума сабирака S и процена грешке $S2$ на нулу (неутрални елемент за сабирање). Затим се редом, у свакој итерацији, вредност YI (елемент низа) додаје на процену грешке. У привременој променљивој T се чува збир суме и $S2$ (односно процене грешке збирно са новим елементом), што заправо представља нову, привремену укупну суму сабирака, која садржи одређену грешку приликом заокруживања. Након тога се рачуна нова

процена грешке $(S-T) + S^2$, што представља главну идеју алгоритма. У идеално прецизним условима вредност овог израза би увек била нула. Ипак, пошто су у питању бројеви у покретном зарезу, присутна је грешка приликом заокруживања. На овај начин се рачуна управо то одступање. На крају се сума постави на тренутну вредност збира, па се поступак понавља док се не заврше сви елементи низа. Овим поступком се грешка настала заокруживањем не акумулира линеарно, већ се у сваком кораку делимично исправља, чиме се тачност резултата значајно побољшава.

ii) Модерна верзија Кахановог алгоритма

У савременој литератури, Каханов алгоритам се представља у нешто другачијем облику. Ова верзија је под уобичајеним претпоставкама заокруживања (*roundTiesToEven*) и у одсуству изузетних ситуација, као што су *overflow* или *NaN* математички еквивалентна оригиналној. Редослед операција је промењен, тако да се грешка приликом заокруживања компензује пре него што се нови елемент дода на суму. Такође знак корекције грешке је супротан у односу на оригиналну верзију. Приказ датог алгоритма у форми псеудокода дат је у наставку.

```
S = X[1]
C = 0
for j = 2 to N {
    Y = X[j] - C
    T = S + Y
    C = (T - S) - Y
    S = T
}
```

Код 3.2.1.2. Каханов алгоритам – псеудокод [2]

У поређењу са оригиналним обликом разлика је одмах у првој линији кода где се коначан збир S одмах поставља на вредност првог елемента низа X , уместо на нулу, па се број итерација петље смањује за један. Операције унутар петље су у овој верзији интуитивније. Одмах се рачуна исправљена вредност новог елемента низа Y , тако што се од њега одузима претходна процена грешке C . Такође се збир чува у привремену променљиву T , чија се вредност на крају исте итерације додељује укупној суми. Рачунање процене грешке C се ради на сличан начин као оригинална верзија. Новији приступ је постао стандардан, јер је лакши за разумевање, а резултати су идентични.

Приликом коришћења наивног сумирања, максимална грешка је реда величине $N\epsilon\sum|x_i|$, односно линеарно зависи од броја елемената низа. Са друге стране, применом Кахановог

алгоритма грешка је ограничена на ред величине $(2\epsilon + O(N\epsilon^2))\sum |x_i|$. Овим се показује да Каханов алгоритам постиже драстично побољшање, а та разлика постаје израженија повећањем величине низа. [2]

iii) Каханов алгоритам у паралелном окружењу

У раду „Performance analysis of the Kahan-enhanced scalar product on current multicore processors” [4] анализирана је перформанса Кахановог алгоритма применом ECM (*execution-cache-memory*) модела, који омогућава прецизну идентификацију уских грла приликом извршавања на нивоу једног језгра, као и предвиђање понашања, при паралелном извршавању на више језгара. Показано је да, чим се подаци налазе ван L1 кеша, време потребно за пренос података из меморије постаје веће од времена потребног за рачунање, услед чега су и наивно и Каханово сабирање подједнако ограничени временом приступања меморији. Из тог разлога, додатне аритметичке операције које уводи Каханов алгоритам практично не утичу на укупно време извршавања. Ово важи под условом да је код векторизован. Ипак уколико се користи скаларна (невекторизована) имплементација, Каханов алгоритам је приметно спорији од наивног, пошто је у том случају ограничавајући фактор број аритметичких операција, а не приступ меморији. Аутори такође истичу да преводиоци, приликом агресивне оптимизације кода, могу закључити да је компензациона променљива увек једнака нули, чиме се Каханов алгоритам ефективно своди на наивно сабирање.

Битно је напоменути да се закључак да је Каханов алгоритам готово бесплатан искључиво односи на одговарајуће оптимизоване SIMD имплементације, које укључују и ручно одмотавање петље (*loop unrolling*) и на архитектуру коју су аутори анализирали, не и на скаларни C код, попут онога коришћеног у имплементацији представљеној у овом раду. Такође, аутори раздвајају понашање приступа L2/L3 кеша, где је овај закључак примарно потврђен од приступа главној (RAM) меморији.

3.2.2. Нојмајеров алгоритам

Нојмајеров алгоритам користи исту идеју као Каханов алгоритам, али је додатно побољшава. Проблем у Кахановом алгоритму настаје када је нови елемент низа већи од суме. У том случају није обезбеђено да се остатак поуздано издваја у поретку који Каханов алгоритам користи, иако не значи да Каханов алгоритам нужно погрешно израчунава грешку у сваком оваквом кораку. Због тога Нојмајеров алгоритам врши проверу да ли је по

апсолутној вредности тренутна сума већа од новог елемента низа. Уколико јесте, корекција се рачуна на исти начин као и у Кахановом алгоритму, а у супротном улоге суме и новог елемента се замене. За разлику од Кахановог алгоритма, ова грешка се не користи при рачунању суме у свакој итерацији, већ само на крају, када се изађе из петље. Приказ датог алгоритма у форми псеудокода дат је у наставку.

```
S = X[1]
C = 0
for j = 2 to N {
    T = S + X[j]
    if (|S| >= |X[j]|) {
        C = C + ((S - T) + X[j])
    } else {
        C = C + ((X[j] - T) + S)
    }
    S = T
}
S = S + C
```

Код 3.2.2.1. Нојмајеров алгоритам – псеудокод

Нојмајер је у свом раду, поред описа алгоритма доказао да овако побољшани алгоритам поседује строжу границу грешке (реда величине $(0.75N^2 + N)\epsilon^2 \sum |x_i|$) у поређењу са основним Кахановим алгоритмом, односно побољшава тачност резултата. Битно је напоменути да ова граница представља теоријски горњу границу грешке, а не тврдњу да Нојмајеров алгоритам за сваки појединачни улаз мора дати мању грешку од Кахановог.[5]

4. ПРЕГЛЕД КОРИШЋЕНЕ ТЕХНОЛОГИЈЕ И ТЕСТ ОКРУЖЕЊА

У овом поглављу биће представљени OpenMP, који се примењује за паралелну имплементацију, спецификација платформе на којој је вршено тестирање, као и три тест апликације коришћене за проверу прецизности и убрзања.

4.1. OpenMP

OpenMP (*Open Multi-Processing*) је API (апликативни програмски интерфејс) који омогућава једноставно писање паралелних апликација на системима са дељеном меморијом. Његова три главна дела су преводилачке директиве, рутине извршне библиотеке и променљиве окружења. Имплементиран је по принципу *fork-join* паралелизма, где главна нит покреће скуп радних нити за паралелни део кода, које се по завршетку поново спајају у главну нит.

На језицима C/C++ се за OpenMP користи директива облика `#pragma omp` изнад дела кода који је потребно паралелизовати. У случају да се превођење врши без одговарајућих компајлерских опција, онда се код извршава секвенцијално, а директиве игноришу. Како би се код извршавао паралелно и креирала група нити које истовремено извршавају исти блок кода додаје се кључна реч *parallel* - `#pragma omp parallel`. Директива `#pragma omp parallel for` статички или динамички расподељује итерације петље по нитима. За одређивање укупног броја нити и редног броја нити која се тренутно извршава се користе функције `omp_get_num_threads()` и `omp_get_thread_num()`, док се за мерење протеклог времена примењује функција `omp_get_wtime()`.

Проблем који може настати приликом аритметичких операција великог броја елемената низа је истовремени приступ истој меморијској локацији у којој је смештен резултат. Из тог разлога OpenMP уводи клаузулу *reduction*, чијом применом свака нит креира приватну копију наведене променљиве и локално рачуна свој парцијални збир. По завршетку паралелног региона, сви парцијални резултати се аутоматски комбинују наведеном операцијом у коначну вредност. На овај начин се може применити наивно сабирање, а како је

циљ искористити алгоритме за стабилно сумирање, неопходно је креирати корисничке редукционе операције за сваки алгоритам. [6]

4.2. Тест платформа

Тестирање је извршено на удаљеном серверу rtidev5, развојној машини Електротехничког факултета. На њему су инсталирани алати и одговарајуће библиотеке за рад са OpenMP, као и одговарајуће верзије GCC/G++ преводиоца и линкера [7]. Превођење свих тест апликација извршено је идентичном командом: `gcc -fopenmp -O3 <fileName>.c -o <programName> -lm -lquadmath`. Релевантне спецификације сервера приказане су у табели 4.2.1.

Табела 4.2.1. Спецификације сервера

Процесор	
Модел	Intel Core i7-11700F @ 2.50 GHz (11th Gen)
Број физичких језгара	8
Број логичких језгара (нити)	16 (Hyper-Threading, 2 нити по језгру)
Максимална фреквенција	4.90 GHz
Кеш меморија	
L1 (подаци)	384 KiB
L1 (инструкције)	256 KiB
L2	4 MiB
L3	16 MiB
Систем	
RAM	62 GiB
Оперативни систем	Ubuntu 22.04.5 LTS (Jammy Jellyfish)
Верзија кернела	Linux 5.15.0-186-generic
Софтвер	
Преводацац	GCC 11.4.0
OpenMP верзија	4.5 (201511)

4.3. Тест апликације

За проверу прецизности и убрзања алгоритама коришћена су три програма различите сложености. Најједноставнији је сумирање низа, па множење матрица, а као

најкомпликованији одабрана је молекуларна динамика. За сваки од њих биће употребљене по три тест примера различите сложености.

4.3.1. Сумирање низа (*sumArray*)

Програм *sumArray* врши сабирање свих елемената низа типа *double* и представља најједноставнији програм на који се могу применити алгоритми за стабилно сумирање. У наставку приказано је језгро секвенцијалне имплементације:

```
double s = 0.0;
for (long i = 0; i < N; i++) s += X[i];
```

Код 4.3.1.1. Језгро секвенцијалне имплементације сумирања низа

4.3.2. Множење матрица (*matMul*)

Програм *matMul* врши множење две квадратне матрице типа *double*. Након израчунавања, неопходно је упоредити прецизност за сваки од добијених елемената резултујуће матрице. У наставку приказано је језгро секвенцијалне имплементације:

```
for (long i = 0; i < N; i++) {
    for (long j = 0; j < N; j++) {
        double sum = 0.0;
        for (long k = 0; k < N; k++) {
            sum += A[i * N + k] * B[k * N + j];
        }
        C[i * N + j] = sum;
    }
}
```

Код 4.3.2.1. Језгро секвенцијалне имплементације множења матрица

4.3.3. Молекуларна динамика (*md*)

Програм *md* симулира честице које међусобно интерагују зависно од удаљености (*N-body* проблем). На основу познатих позиција и брзина честица у датом тренутку израчунавају се нове вредности за следећи тренутак. Укупна сила се рачуна као сума доприноса свих осталих честица у систему. Због тога што се симулација извршава кроз већи број временских корака, грешке настале приликом заокруживања се додатно акумулирају. Изворни код је преузет из извора [7], одељка домаћи задаци, директоријум 2025-2026.

5. ОПИС ИМПЛЕМЕНТАЦИЈЕ

У овом поглављу описани су детаљи имплементације Кахановог и Нојмајеровог алгоритма унутар паралелног окружења OpenMP. Прво су представљени заједнички аспекти решења који се примењују за све три тест апликације, а затим су описане специфичности реализације појединачних апликација.

5.1. Заједнички аспекти имплементације тест апликација

За сваку од тест апликација било је неопходно одредити референтну вредност високе тачности са којом би се касније поредиле секвенцијалне и паралелне верзије наивног, Кахановог и Нојмајеровог сабирања. Због тога је коришћен тип података `__float128`, проширени тип који подржава GCC преводилац на наведеној платформи, док заглавље `quadmath.h` из библиотеке `libquadmath` обезбеђује одговарајуће функције и декларације за рад са овим типом, не уводећи сам језички тип. Овај тип података пружа четвороструку тачност (128 бита), где знак заузима 1 бит, поље експонента 15, а разломљена мантиса 112 бита, што укупно даје прецизност од 113 битова [8]. Ово представља двоструко већу ширину података у односу на тип `double` (64 бита, распоређених у формату 1+52+11 за мантису и експонент). Референтна вредност, иницијално срачуната са четвороструком тачношћу, се заокружује на тип `double` пре фазе поређења. Овако добијени податак представља најближу могућу репрезентацију тачног резултата унутар овог типа података, односно најпрецизнију вредност коју могу постићи алгоритми. У наставку је приказан део кода коришћен унутар тест апликације за сумирање низа, док се еквивалентно решење примењује и за комплексније апликације.

```
__float128 ref_sum = 0.0Q;  
for (long i = 0; i < N; i++) {  
    ref_sum += (__float128)X[i];  
}  
double ref_double = (double)ref_sum;
```

Код 5.1.1. Рачунање референтне суме на примеру сумирања низа

С обзиром на то да се у оквиру Кахановог и Нојмајеровог алгоритма истовремено обрађују две вредности (сума и грешка), креирана је структура *KahanAcc* са два одговарајућа поља *sum* и *corr* типа *double*. Оваква структура омогућава да се као повратна вредност функција врате оба података, што представља предуслов за имплементацију кориснички дефинисаних редукција ова два алгоритма за стабилно сумирање.

```
typedef struct {
    double sum;
    double corr;
} KahanAcc;
```

Код 5.1.2. Дефиниција структуре *KahanAcc*

Дефинисана је по једна функција за Каханов и Нојмајеров алгоритам која се примењује над структуром *KahanAcc* и вредношћу типа *double* коју је потребно додати на суму. На основу логике алгоритма, функција враћа нову вредност суме и корекцију грешке. Обе функције су дефинисане као *inline*, чиме се преводиоцу сугерише, али не и гарантује, уграђивања тела функције на месту позива у циљу смањења режијских трошкова позива функције.

```
static inline KahanAcc kahan_update(KahanAcc acc, double value) {
    double y = value - acc.corr;
    double t = acc.sum + y;
    acc.corr = (t - acc.sum) - y;
    acc.sum = t;
    return acc;
}

static inline KahanAcc neumaier_update(KahanAcc acc, double value)
{
    double t = acc.sum + value;
    if (fabs(acc.sum) >= fabs(value)) {
        acc.corr += (acc.sum - t) + value;
    } else {
        acc.corr += (value - t) + acc.sum;
    }
    acc.sum = t;
    return acc;
}
```

Код 5.1.3. Функције за Каханово и Нојмајерово сумирање

Декларисане су кориснички дефинисане редукције за оба алгоритма, које користе раније дефинисану структуру. За почетне вредности се поставља 0.0 као неутрални елемент за операцију сабирања. Позивају се одговарајуће функције за Каханов, односно Нојмајеров

алгоритам и на тај начин се прорачунава сума. На израчунату акумулирану грешку унутар функције се додаје и вредност грешке другог аргумента редукције. Тиме је осигурано да се приликом спајања парцијалних резултата различитих нити не изгуби корекција грешке која је израчуната за сваку нит.

```
#pragma omp declare reduction(kahanRed : KahanAcc : \
    omp_out = kahan_update(omp_out, omp_in.sum), \
    omp_out.corr += omp_in.corr) \
    initializer(omp_priv = {0.0, 0.0})

#pragma omp declare reduction(neumaierRed : KahanAcc : \
    omp_out = neumaier_update(omp_out, omp_in.sum), \
    omp_out.corr += omp_in.corr) \
    initializer(omp_priv = {0.0, 0.0})
```

Код 5.1.4. Декларација кориснички дефинисаних редукција са Кахановим и Нојмајеровим сумирањем

Због потребе за тестирањем са великим скуповима података, низовима и матрицама са великим бројем елемената, имплементиран је механизам за генерисање и учитавање података из датотека. На овај начин се осигурава да се врши прорачун над идентичним улазним вредностима за све алгоритме, чиме се омогућава објективно поређење.

5.2. Специфичности имплементације тест апликација

У овом потпоглављу биће изложени специфични детаљи имплементације појединачних тест апликација који нису обухваћени претходним описом заједничких елемената. С обзиром на то да апликација за сумирање низа представља директну примену редукционих оператора описаних у поглављу 5.1, она неће бити предмет посебне анализе у овом делу. У наставку је фокус стављен на специфичности паралелизације множења матрица и симулације молекуларне динамике.

5.2.1. Специфичности множења матрица

У оквиру апликације за множење матрица, за паралелно рачунање елемената резултујуће матрице коришћена је директива *#pragma omp parallel for collapse(2)*. Она омогућава да се два нивоа угњеждености петљи посматрају као јединствена целина и на тај начин побољшава паралелизацију. Пошто је број врста и колона велики, рачуна се велики број међусобно независних елемената. Због тога свака нит независно од осталих може рачунати појединачне елементе. Додатно паралелизовање унутрашње петље (са бројачем k) би захтевало додатну синхронизацију, што би успорило извршавање, па у оквиру ове

апликације није примењен механизам редукције. У наставку је приказан део кода са описаном паралелизацијом и Кахановим сумирањем.

```
static void matmul_parallel_kahan(const double *A, const double *B,
double *C, long N) {
    #pragma omp parallel for collapse(2)
    for (long i = 0; i < N; i++) {
        for (long j = 0; j < N; j++) {
            KahanAcc acc = {0.0, 0.0};
            for (long k = 0; k < N; k++) {
                acc = kahan_update(acc, A[i * N + k] * B[k * N + j]);
            }
            C[i * N + j] = acc.sum + acc.corr;
        }
    }
}
```

Код 5.2.1.1. Функција за паралелно множење матрице применом Кахановог алгоритма

5.2.2. Специфичности молекуларне динамике

За разлику од претходне две апликације, ова је комплекснија због зависности од претходног временског тренутка. Позиције и брзине честица ажурирају се тек након што се силе израчунају за све честице за дати временски корак.

```
KahanAcc total_pe = {0.0, 0.0};
KahanAcc total_ke = {0.0, 0.0};
#pragma omp parallel for reduction(kahanRed: total_pe, total_ke)
for (int k = 0; k < np; k++) {
    KahanAcc force_acc[3] = {{0.0, 0.0}, {0.0, 0.0}, {0.0, 0.0}};
    ...
    for (int j = 0; j < np; j++) {
        ...
        total_pe = kahan_update(total_pe, 0.5 * sin(d2) * sin(d2));
        for (int i = 0; i < nd; i++) {
            double fterm = -rij[i] * sin(2.0 * d2) / d;
            force_acc[i] = kahan_update(force_acc[i], fterm);
        }
    }
    for (int i = 0; i < nd; i++) {
        f[i + k * nd] = force_acc[i].sum + force_acc[i].corr;
        total_ke = kahan_update(total_ke, vel[i + k * nd] * vel[i + k * nd]);
    }
}
*pot = total_pe.sum + total_pe.corr;
*kin = 0.5 * mass * (total_ke.sum + total_ke.corr);
}
```

Код 5.2.2.1. Рачунање енергија и силе применом паралелне верзије Кахановог алгоритма

Прати се вредност потенцијалне и кинетичке енергије, коришћењем одговарајуће верзије редукције, слично као и код сумирања низа. Са друге стране рачунање силе се врши

директно применом одговарајуће функције, као код множења матрица. У наставку је приказан део кода паралелне верзије прорачуна применом Кахановог алгоритма

6. РЕЗУЛТАТИ И ДИСКУСИЈА

У овом поглављу биће представљени резултати добијени покретањем три тест апликације различите сложености. У зависности од броја нити (1, 4, 8 или 16) и коришћеног алгоритма посматраће се разлика у убрзањима и нумеричкој стабилности. Време извршавања мерено је функцијом *omp_get_wtime()*, при чему је обухваћена искључиво временска петља, без алокација, иницијализације, читања из датотеке и рачунања референтних вредности.

6.1. Резултати програма за сумирање низа

Сумирање низа представља најједноставнији проблем, па је за њега било неопходно генерисати довољно велике тест примере како би се боље уочила разлика између алгоритама. Генерисане су три датотеке од којих прва садржи 10.000.000 елемената, а друге две датотеке по 100.000.000 елемената низа. Такође дефинисана је расподела појављивања бројева из одговарајућих опсега, како би постојала разноликост реда величине бројева. Расподела је представљена у табели 6.1.1, а у наставку је дат пример *Python* скрипте за креирање првог примера.

Табела 6.1.1. Карактеристике тест примера за сумирање низа

Тест пример	N	Расподела
mixed_random_10m	10.000.000	60% (-1,1); 10% (-10,10); 20% (-100,100); 10% (-1000,1000)
mixed_random_100m	100.000.000	20% (-0.1,0.1); 40% (-1,1); 10% (-10,10); 20% (-100,100); 10%(-1000,1000)
mixed_100m_log_ranges	100.000.000	По $\sim 11.11\%$ (односно $\frac{1}{9}$) за 9 интервала облика $(-10^k, 10^k)$ за $k = -4, \dots, 4$

Сваки од три наведена тест примера покренут је са различитим бројем нити: 1, 4, 8 и 16 по пет пута и одређене су просечне вредности за време извршавања, апсолутне и релативне грешке. На сликама 6.1.1-6.1.3. су приказани графици зависности релативне грешке од броја нити и алгоритма, с тим да је релативна грешка приказана помоћу логаритма, јер се ред величине разликује зависно од алгоритма. Из разлога што је грешка Нојмајеровог сабирања за сваки од три тест примера једнака нули, њена вредност на скали се математички не може представити стубићем, па је експлицитно означена маркером.

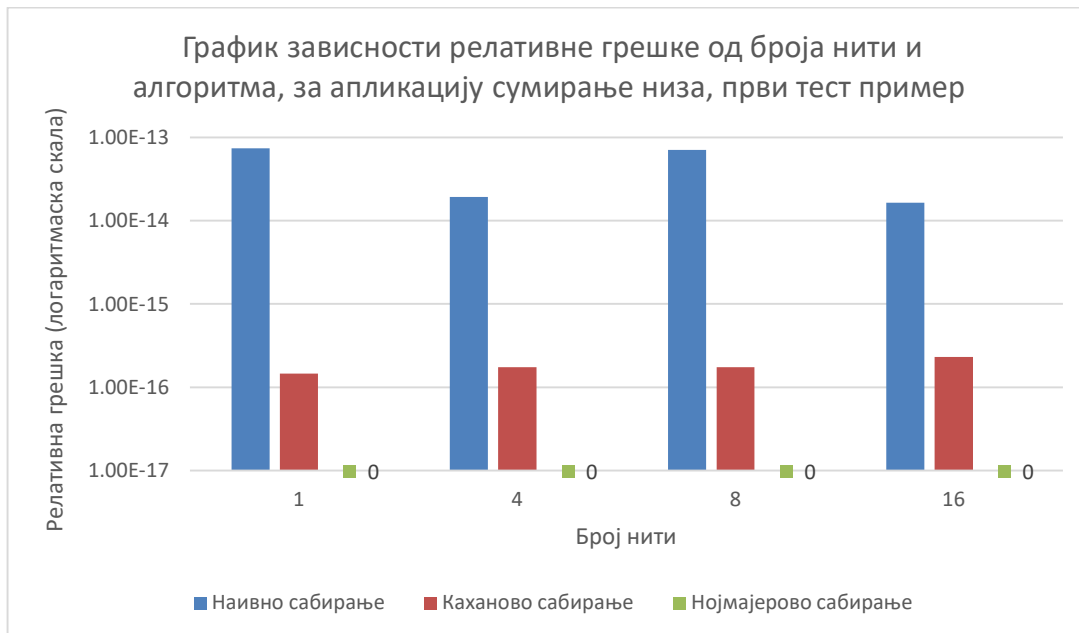
```

import random
n = 10_000_000
path = 'mixed_random_10m.txt'
with open(path, 'w') as f:
    f.write(f'{n}\n')
    for i in range(n):
        r = random.random() * 100.0
        if r < 60.0:
            v = random.uniform(-1.0, 1.0)
        elif r < 70.0:
            v = random.uniform(-10.0, 10.0)
        elif r < 90.0:
            v = random.uniform(-100.0, 100.0)
        else:
            v = random.uniform(-1000.0, 1000.0)
        f.write(f'{v:.17g}\n')

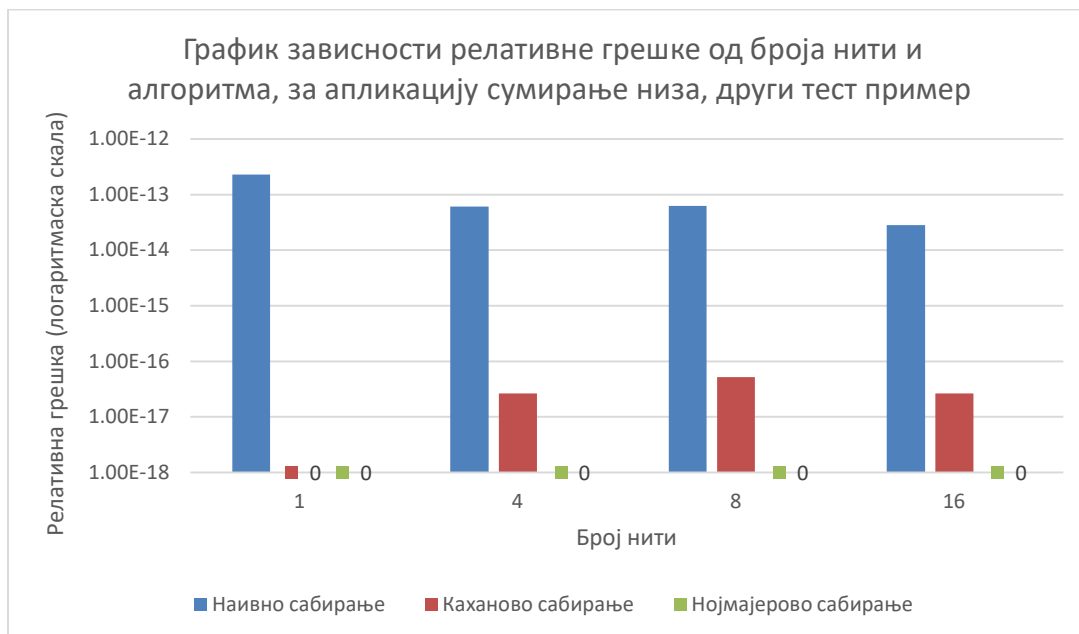
```

Код 6.1.1. Скрипта за генерисање датотеке за тест пример

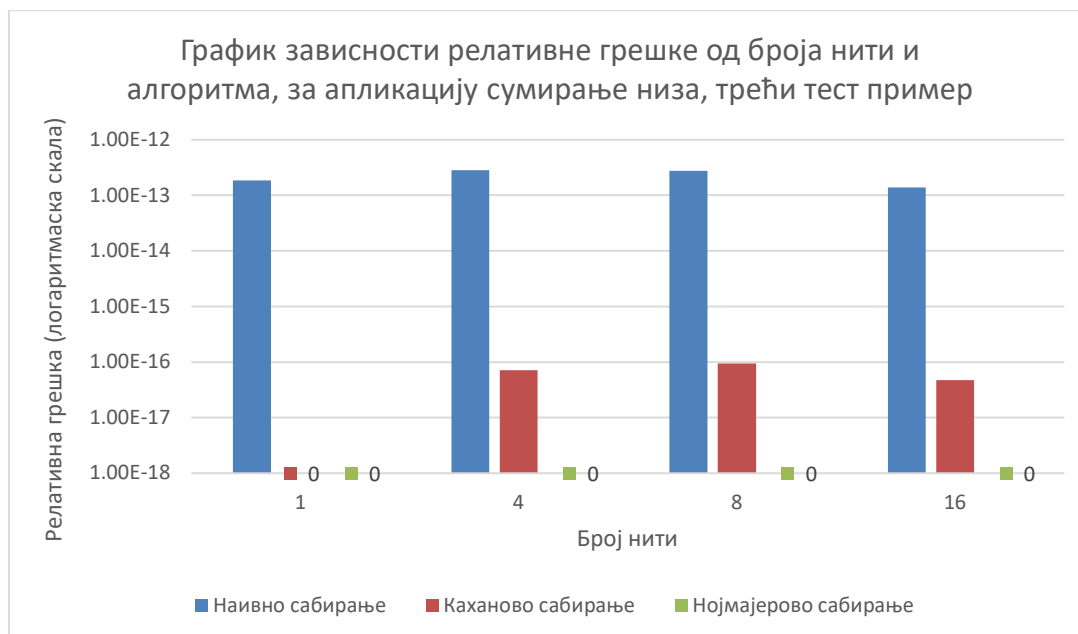
У сва три тест примера Каханово и Нојмајерово сумирање показују значајно већу прецизност у односу на наивно сабирање, при чему се Нојмајерово сабирање у овим тестовима подудара са референтном заокруженом на формат *double*. Поређење у односу на вредност са четвороструком тачношћу за Нојмајеров алгоритам доводи до релативне грешке реда величине 10^{-17} . Ипак такво поређење није толико релевантно, пошто таква грешка потиче од ограничења саме *double* репрезентације броја, односно најпрецизнију вредност није ни могуће приказати у овом формату.



Слика 6.1.1. График зависности релативне грешке од броја нити и алгорита, за апликацију сумирање низа, први тест пример



Слика 6.1.2. График зависности релативне грешке од броја нити и алгоритма, за апликацију сумирање низа, други тест пример

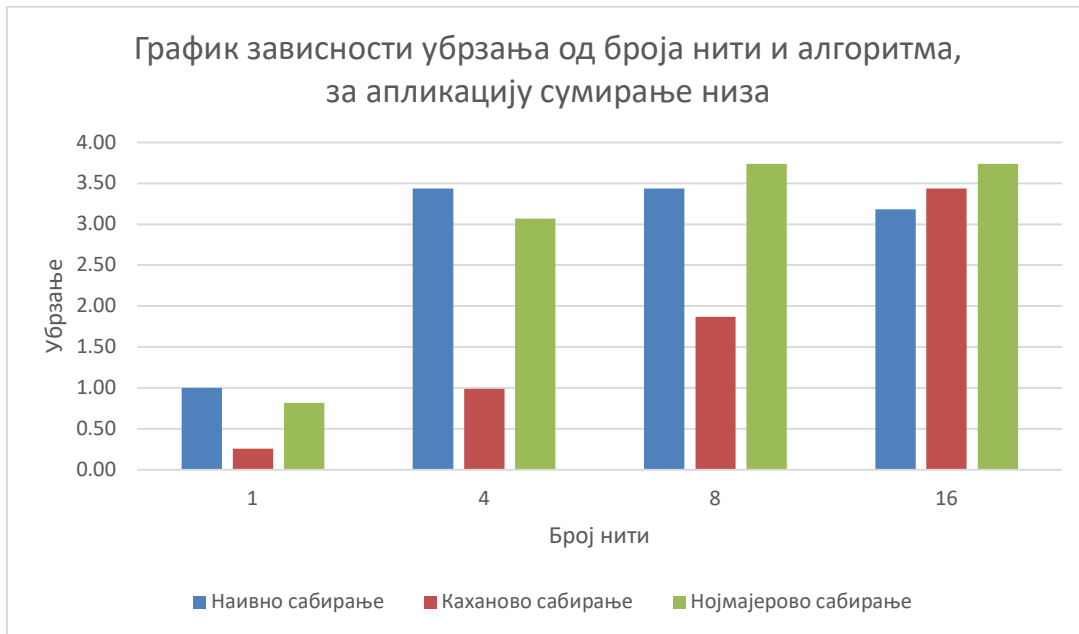


Слика 6.1.3. График зависности релативне грешке од броја нити и алгоритма, за апликацију сумирање низа, трећи тест пример

На слици 6.1.4. приказан је график убрзања у зависности од броја нити за сва три алгоритма за тест пример *mixed_random_10m*, јер су убрзања за сва три тест примера јако слична. Убрзање је рачунато као однос $T_{naive}(1)/T_a(p)$, где $T_{naive}(1)$ представља време

извршавања серијског наивног алгоритма, а $T_a(p)$ време извршавања варијанте a (наивно, Каханово или Нојмајерово сабирање) са p нити. Може се уочити да Нојмајерово сабирање постиже до око три пута веће убрзање од Кахановог. С обзиром на то да Нојмајеров алгоритам садржи додатну условну проверу у односу на Каханов очекивано би било да Каханов алгоритам буде бржи од Нојмајеровог, међутим резултати показују супротно. Ова појава вероватно потиче од начина на који преводилац оптимизује код ових алгоритама, где Нојмајеров алгоритам има више инструкција које нису зависне од претходних, па се потенцијално боље испољава паралелизам на нивоу инструкција.

Такође се примећује да се повећањем броја нити, убрзања за сва три алгоритма све више приближавају један другом и спорије увећавају. Могућ узрок томе је да већи део времена одлази на читања података из меморије (RAM, не датотеке, јер читање из датотеке не улази у измерено време) него на само извршавање неопходних аритметичких операција.



Слика 6.1.4. График зависности убрзања од броја нити и алгоритма, за апликацију сумирање низа за тест пример `mixed_random_10m`

6.2. Резултати програма за множење матрица

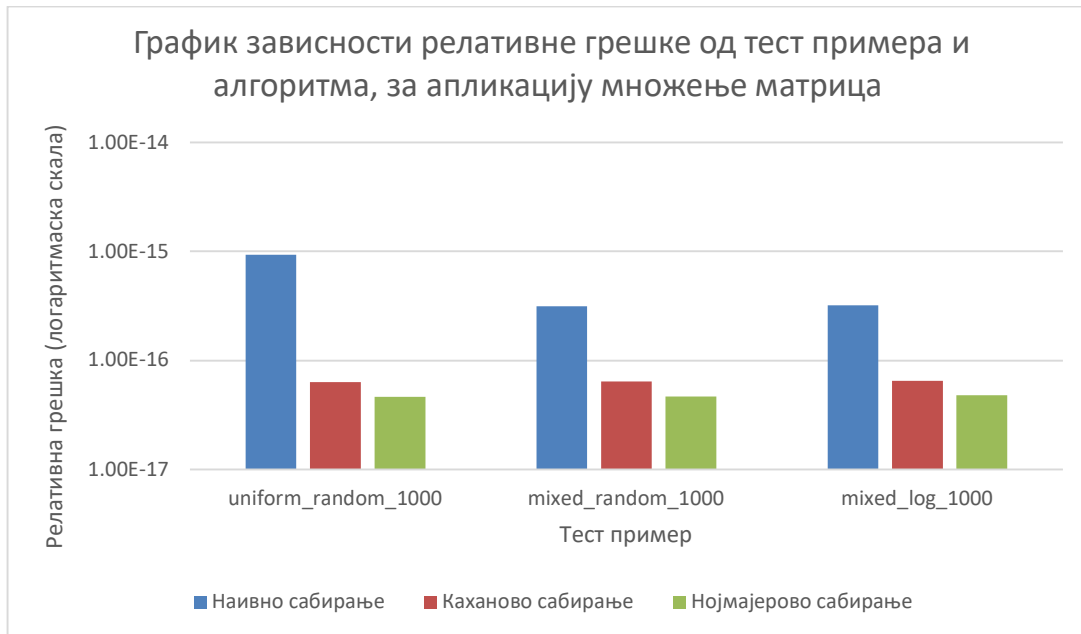
За тестирање апликације за множење матрица коришћене су шест матрица димензија 1000x1000, по две за сваки од тест примера, чији су елементи генерисани по расподелама приказаним у табели 6.2.1. Сваки од наведених тест примера покретан је са различитим

бројем нити: 1, 4, 8 и 16 по пет пута, при чему су одређене просечне вредности за апсолутну и релативну грешку, максимално одступање појединачних елемената резултујуће матрице и време извршавања.

Табела 6.2.1. Карактеристике тест примера за множење матрица

Тест пример	Димензије	Расподела
uniform_random_1000	1000x1000	Униформно из (-1,1)
mixed_random_1000	1000x1000	60% (-1,1); 10% (-10,10); 20% (-100,100); 10% (-1000,1000)
mixed_log_1000	1000x1000	По $\sim 11.11\%$ (односно $\frac{1}{9}$) за 9 интервала облика $(-10^k, 10^k)$ за $k = -4, \dots, 4$

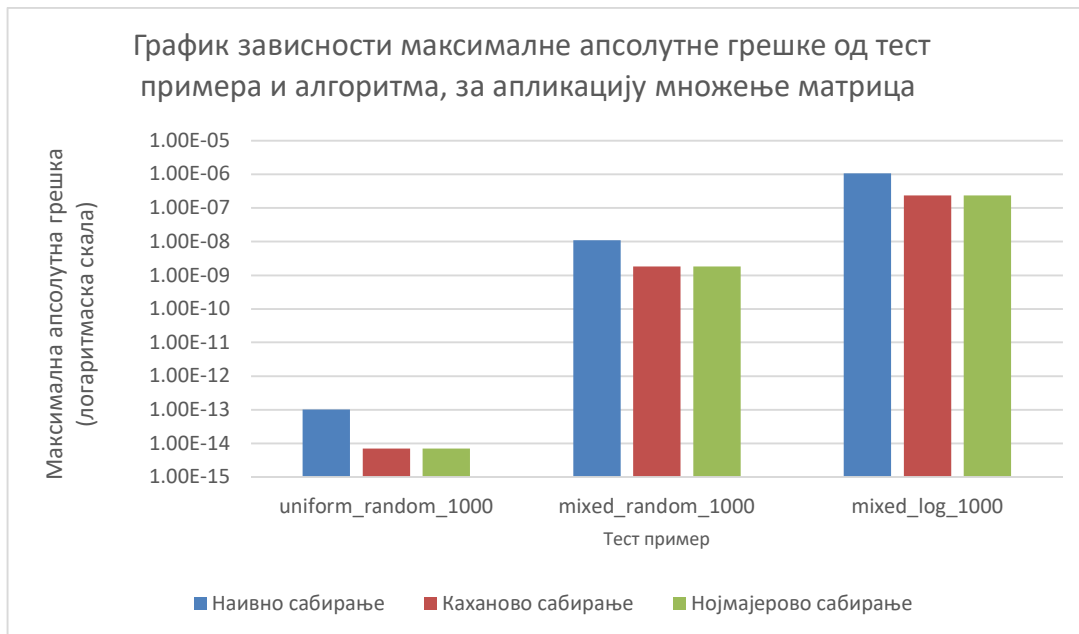
Пошто приликом паралелизације множења матрица свака нит независно рачуна вредност појединачног елемента резултујуће матрице, релативна грешка не зависи од броја нити. Због тога су на слици 6.2.1. приказане грешке у зависности од алгоритма и тест примера, а не од броја нити. Глобална релативна грешка израчуната је тако што је сума апсолутних вредности грешке сваког елемента матрице подељена са сумом апсолутних вредности референтне матрице. Са истом зависношћу, праћена је и максимална апсолутна грешка појединачног елемента резултујуће матрице, представљена на слици 6.2.2. Посматрана метрика представља највеће измерено одступање у посматраној матрици.



Слика 6.2.1. График зависности глобалне релативне грешке од тест примера и алгоритма, за апликацију множење матрица

Каханов и Нојмајеров алгоритам компензују грешку сабирања, али не и грешку која настаје приликом операције множења сваког појединачног елемента. Та грешка настаје при

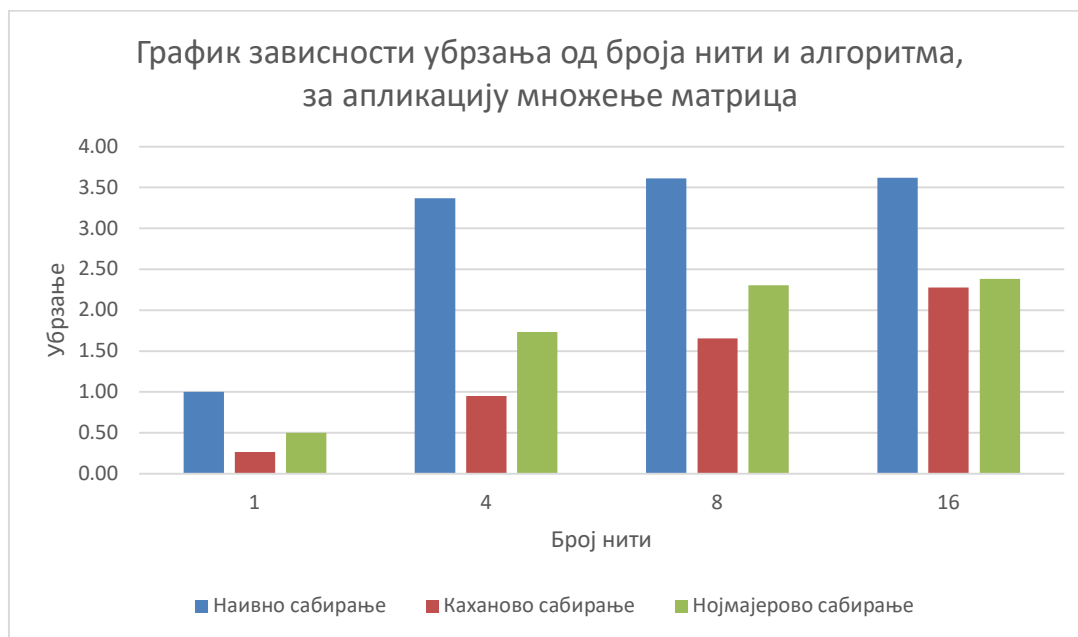
заокруживању међурезултата и не може се исправити алгоритмима за сумирање. На слици 6.2.2. се може уочити да су максималне апсолутне грешке за Каханово и Нојмајерово сабирање идентичне и знатно мање него код наивног сумирања, што је могућ показатељ да доминантни извор грешке потиче управо од операције множења, али не и доказ тога. Ипак на слици 6.2.1. се примећује да је укупна релативна грешка мања за Нојмајерово сабирање. Могућ узрок томе је то што иако је грешка настала приликом множења иста за оба алгоритма, приликом сабирања парцијалних производа унутар сваког елемента, у случају када је нови сабирак већи од суме, Нојмајеров алгоритам може постићи нешто већу прецизност.



Слика 6.2.2. График зависности максималне апсолутне грешке елемента матрице од тест примера и алгоритма, за апликацију множење матрица

На слици 6.2.3. приказан је график убрзања у зависности од броја нити за тест пример *mixed_random_1000*. Изабран је као илустративни пример, јер се по понашању не разликује значајно од преостала два. Убрзање је рачунато као однос $T_{naive}(1)/T_a(p)$, где $T_{naive}(1)$ представља време извршавања серијског наивног алгоритма, а $T_a(p)$ време извршавања варијанте a (наивно, Каханово или Нојмајерово сабирање) са p нити. У свим тест примерима најбрже је наивно, затим Нојмајерово, а најспорије Каханово сумирање. Времена, па самим тим и убрзања за наивно сумирање су готово идентична за сва три тест примера, што је случај и за Каханово. Код Нојмајерог алгоритма повећањем реда величине у расподели

вредности време извршавања се мало продужава. Претпоставка је да се то дешава зато што је услов да је сума већа од новог елемента са којим се сабира све ређе испуњен.



Слика 6.2.3. График зависности убрзања од броја нити и алгоритма, за апликацију множење матрица

6.3. Резултати програма молекуларна динамика

За тестирање апликације молекуларне динамике коришћена су три тест примера различите сложености, приказана у табели 6.3.1, која се разликују по броју просторних димензија, броју честица у систему, броју и величини временских корака симулације. Сваки од наведених тест примера покретан је са различитим бројем нити: 1, 4, 8 и 16 по пет пута, након чега су одређене просечне вредности. Множењем вредности за број и величину временских корака из табеле може се за сваки од тест примера одредити укупан временски хоризонт. Тако за прва два тест примера износи 50 временских јединица, док за трећи 100 временских јединица.

Табела 6.3.1. Карактеристике тест примера за апликацију молекуларна динамика

Тест пример	Број димензија	Број честица	Број временских корака	Величина временског корака
md_2d_500_500	2	500	500	0.1
md_3d_300_500	3	300	500	0.1
md_3d_1000_500	3	1000	500	0.2

Укупна енергија у молекуларној динамици се израчунава у сваком кораку, али је њена грешка, рачуната поређењем са почетном вредношћу енергије, готово идентична за све алгоритме и бројеве нити. Узрок је то што се положаји и брзине честица у сваком кораку мењају коришћењем приближних формула, при чему настаје грешка, која се акумулира током симулације. При томе се, поред сабирања, извршавају операције множења, дељења, кореновања и синуса, за чије се грешке приликом заокруживања претпоставља да су значајно веће у односу на операцију сабирања. Потенцијално, због тога алгоритми за стабилно сумирање немају довољно велики утицај на коначну грешку, па се у даљој анализи разматрају искључиво грешке везане за израчунавање силе.

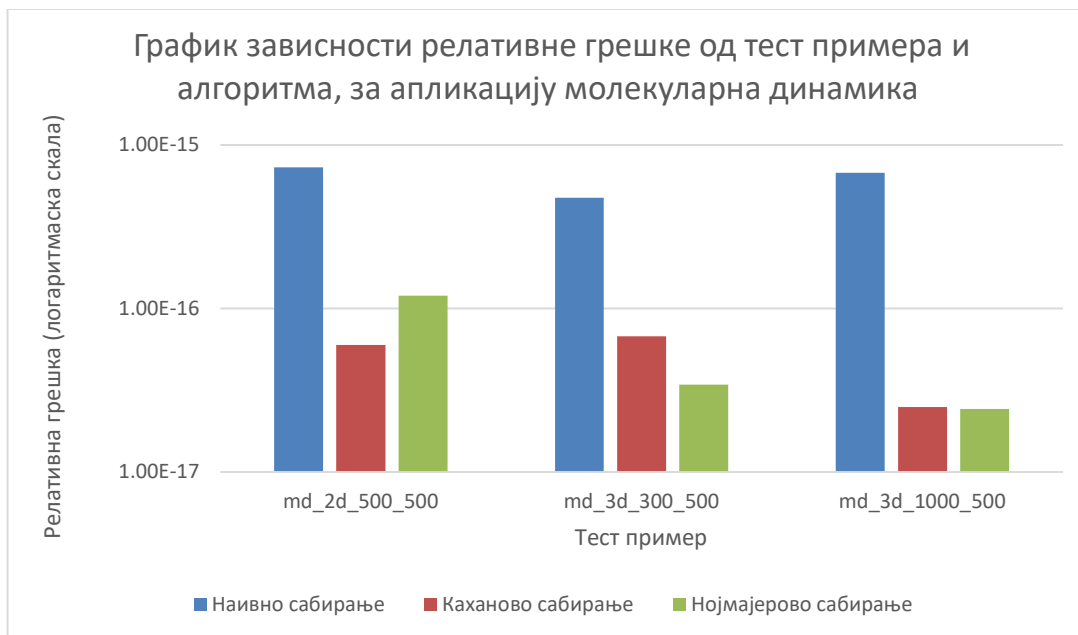
У паралелним деловима кода силе се рачунају независно по честицама, па не постоји потреба за редукционом операцијом. Сабирање се врши директним позивима функције *kahan_update*, односно *neumaier_update*. Редослед сабирања унутар сваке честице је идентичан, без обзира на број нити, па грешка силе не зависи од броја нити.

Како би се смањио утицај других операција, за сваки од алгоритама је посебно рачуната референтна вредност на основу позиција честица тог алгорита. На тај начин је операција сабирања, бар делимично изолована од осталих и добијена јаснија слика о утицају типа сабирања. Такође је извршено поређење са референтном вредношћу добијеном засебним извршавањем симулације са четвороструком тачношћу, која је након тога заокружена на *double* формат. Добијене грешке су, слично као и код енергије, биле готово идентичне независно од алгорита.

На сликама 6.3.1-6.3.3. приказани су графици у логаритамској скали зависности од тест примера и коришћеног алгорита за следеће три метрике, при чему свака користи искључиво вредности сила у последњем кораку:

- 1) глобална релативна векторска грешка свих компоненти силе, која представља глобалну меру тачности и рачуна се по формули
$$VecRelError = \frac{\sum_{k=1}^{np} \sum_{i=1}^{nd} |f_{ref}(i,k) - f_{alg}(i,k)|}{\sum_{k=1}^{np} \sum_{i=1}^{nd} |f_{ref}(i,k)|},$$
 при чему су $f_{ref}(i, k)$ референтна вредност компоненте силе за честицу k и димензију i , $f_{alg}(i, k)$ вредност добијена одговарајућим алгоритмом, np број честица и nd број димензија.

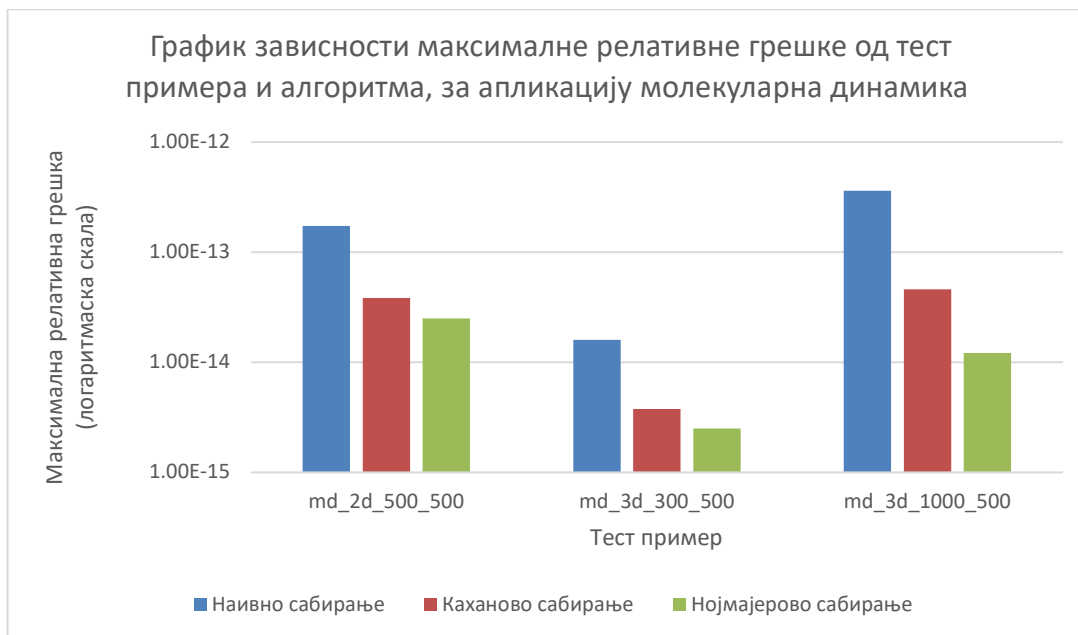
- 2) максимална локална релативна грешка по појединачној компоненти силе, која се рачуна по формули $MaxRelForceDiff = \max_{k,i} \frac{|f_{ref}(i,k) - f_{alg}(i,k)|}{|f_{ref}(i,k)|}$
- 3) сума свих сила на све честице, која према закону акције и реакције мора бити једнака нули.



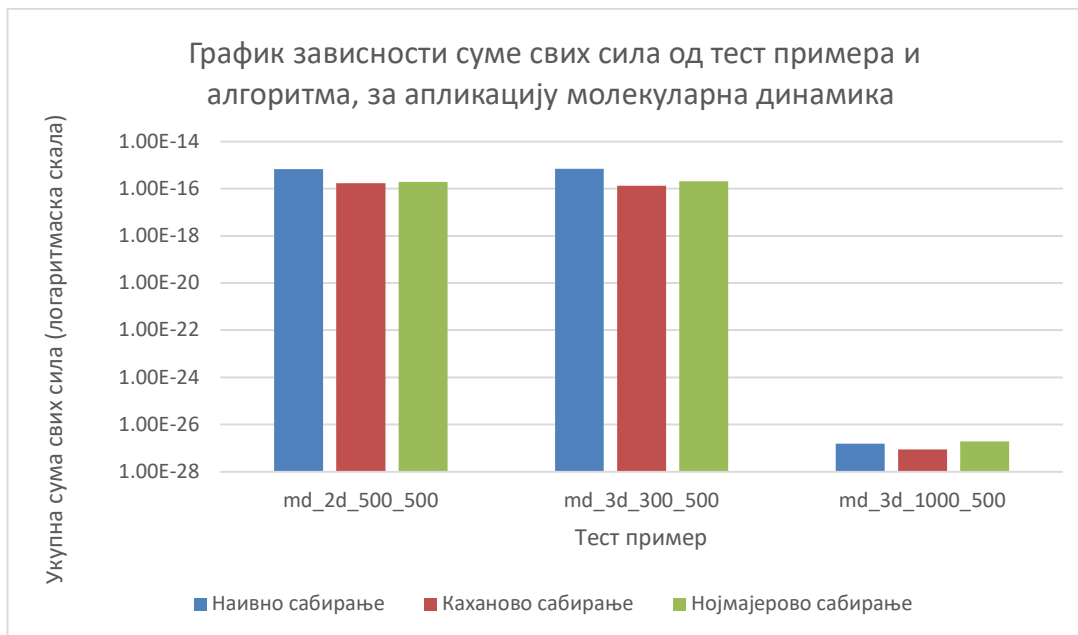
Слика 6.3.1. График зависности глобалне релативне грешке силе од тест примера и алгоритма, за апликацију молекуларна динамика

Наивно сабирање даје знатно већу грешку у односу на Каханово и Нојмајерово сабирање. Међутим, Каханов и Нојмајеров алгоритам дају јако сличне резултате, где су разлике глобалне релативне грешке реда величине од 10^{-16} до 10^{-17} . Поред тога што је разлика мала, од тест примера и метрике зависи који алгоритам ће постићи већу тачност, па се разлика у грешци ова два алгоритма за посматрану апликацију може сматрати практично занемарљивом.

Постоје два потенцијална разлога за ово понашање. Прво, Нојмајерова корекција има предност само када је апсолутна вредност новог сабирка већа од апсолутне вредности тренутне суме, али се у тим случајевима њихов ред величине ретко значајно разликује, па та оптимизација не прави значајну разлику у резултату. Такође, главни извор грешке у израчунавању сила није само сабирање, већ и аритметичке операције множења, дељења и израчунавање синуса над *double* вредностима. Те грешке настају пре самог сабирања, па их алгоритми за стабилно сумирање не могу исправити.



Слика 6.3.2. График зависности максималне релативне грешке од тест примера и алгоритма, за апликацију молекуларна динамика

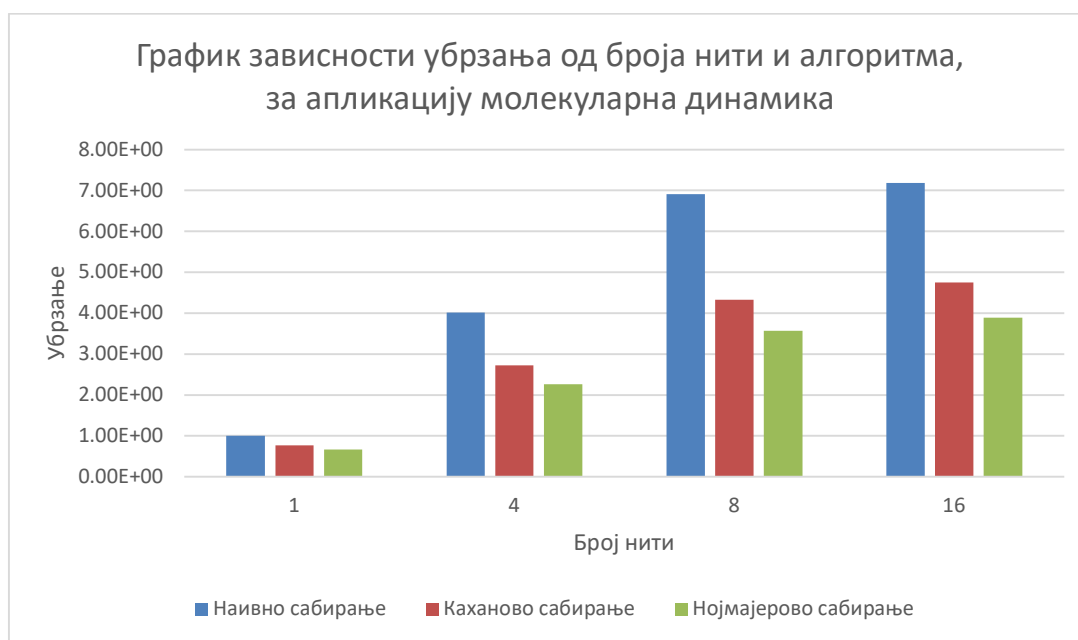


Слика 6.3.3. График зависности укупне суме свих сила на све честице од тест примера и алгоритма, за апликацију молекуларна динамика

За рачунање суме свих сила на све честице коришћено је Нојмајерово сумирање, како би се што прецизније израчунала посматрана метрика. На слици 6.3.3. за тест пример *md_3d_1000_500* вредности су реда величине 10^{-27} , што је значајно мање него код прва два

примера, где су вредности реда 10^{-16} . Разлог томе је да ред величине ове грешке, слично као и код апсолутне грешке, зависи од појединачног примера, па је значај посматране метрике искључиво у поређењу алгоритама, а не тест примера.

Пошто су добијена убрзања готово идентична за све тест примере, на слици 6.3.4. приказан је график зависности убрзања од броја нити и алгорита за тест пример *md_3d_1000_500*, јер је најкомплекснији. Убрзање је рачунато као однос $T_{naive}(1)/T_a(p)$, где $T_{naive}(1)$ представља време извршавања серијског наивног алгорита, а $T_a(p)$ време извршавања варијанте *a* (наивно, Каханово или Нојмајерово сабирање) са *p* нити. Наивно сабирање је најбрже, затим Каханово, па Нојмајерово, што је у складу са бројем рачунских операција. Међутим, у претходним апликацијама (*sumArray* и *matMul*) Нојмајеров алгоритам је имао мање време извршавања од Кахановог. Претпоставља се да је узрок то што су петље у прва два тест примера значајно једноставније, па процесор потенцијално може боље искористити паралелизам на нивоу инструкција. У молекуларној динамици врши се сумирање сила у две или три димензије, кинетичке и потенцијалне енергије, уз велики број других инструкција унутар петље, па је вероватно тај ефекат мање изражен.



Слика 6.3.4. График зависности убрзања од броја нити и алгорита, за апликацију молекуларна динамика, за тест пример *md_3d_1000_500*

Са порастом броја нити расте и убрзање, ипак са 16 нити у односу на 8 повећање је минимално. Вероватан разлог је да процесор поседује 8 физичких језгара, док преосталих 8 настаје применом *Hyper-Threading* технологије, која дели извршне ресурсе истог физичког

језгра између две нити. Пошто је посматрани тест пример веома захтеван, ресурси су веома искоришћени и са 8 нити, па повећање на 16 нити не доноси толику разлику. Уочено је да за 4 нити убрзање паралелног наивног сабирања је за све тест примере око 4 пута, што приближно одговара линеарном убрзању. Могућ узрок овоме је ефекат кеш меморије.

7. ЗАКЉУЧАК

У оквиру овог рада разматран је проблем нумеричке стабилности резултата приликом сабирања бројева у покретном зарезу у паралелном OpenMP окружењу. Услед ограничене прецизности представљања бројева у покретном зарезу и недостатка својства асоцијативности приликом сабирања, редослед извршавања операција, који у паралелном окружењу често није детерминистички, може утицати на добијени резултат, чиме се доводи у питање тачност добијених резултата.

Као решење овог проблема представљена и имплементирана су два алгорита за стабилно сумирање, који прате грешку насталу у сваком кораку сабирања и накнадно је компензују. У циљу примене Кахановог и Нојмајеровог алгорита у паралелном окружењу имплементирани су кориснички дефинисане OpenMP редукције.

Алгоритми за стабилно сабирање су у односу на наивно довели до значајно веће прецизности рачунања. За апликацију сумирања низа Нојмајерово сабирање давало је идентичне резултате као веома прецизна референтна вредност заокружена на формат *double*, док је Каханово сабирање побољшало релативну грешку за приближно два до четири реда величине, па све до потпуног подударана са референтом вредношћу која је заокружена на *double* формат. Код апликација множења матрица и молекуларне динамике, на грешку значајно утичу друге аритметичке операције (множење, дељење, рачунање синуса), па је релативна грешка побољшана за један до два реда величине у односу на наивно сабирање.

Убрзање код сумирања низа и множења матрица повећањем броја нити је ограничено, вероватно услед тога што време извршавања постаје ограничено читањем из меморије, а не бројем аритметичких операција. Услед потенцијално боље испољеног паралелизма на нивоу инструкција, Нојмајерово сабирање показало се нешто бржим од Кахановог, док је наивно сабирање очекивано најбрже. Код молекуларне динамике постиже се значајније убрзање, због захтевнијих аритметичких операција унутар петље. Каханово сабирање постаје брже од Нојмајеровог, вероватно услед мање изражене оптимизације на нивоу инструкција, при већој оптерећености процесора. Уочено је приближно линеарно убрзање за четири нити, што може бити последица бољег искоришћења кеш меморије, док је убрзање са 16 у односу на 8 нити

било мање изражено услед тога што процесор поседује осам физичких језгара, која су услед Hyper-Threading технологије већ у великој мери искоришћена и при осам нити.

На основу свих спроведених тестова за испитане улазе, платформу и приказане имплементације може се закључити да примена алгоритама за стабилно сумирање доноси значајно бољу тачност резултата, по цену нешто споријег извршавања у односу на наивно сабирање. Битно је напоменути да се ово односи на два различита механизма примене за компензацију грешке кроз кориснички дефинисану OpenMP редукцију или изоловане позиве функција по нитима.

Рад отвара неколико праваца за даље унапређење. С обзиром на то да значајни извор грешке, посебно код множења матрица и молекуларне динамике, потиче од других аритметичких операција, могла би се истражити примена сличних техника компензације и на те операције. Такође, пошто се овај рад фокусирао на тачност, а не и на репродуцибилност резултата независно од броја нити, увођење фиксне структуре редукционог стабла, независно од броја нити представља природан правац даље имплементације ка потпуној репродуцибилности резултата.

8. ЛИТЕРАТУРА

- [1] „IEEE Standard for Floating-Point Arithmetic” , *IEEE Std 754-2019*, IEEE, Jul. 2019.
- [2] D. Goldberg, „What Every Computer Scientist Should Know About Floating-Point Arithmetic,” *ACM Computing Surveys*, 23(1): 5–48, Mar. 1991.
- [3] W. Kahan, „Further Remarks on Reducing Truncation Errors,” *Communications of the ACM*, 8(1): 40, Jan. 1965.
- [4] J. Hofmann, D. Fey, J. Eitzinger, G. Hager, and G. Wellein, „Performance analysis of the Kahan-enhanced scalar product on current multicore processors,” *arXiv:1505.02586*, May 2015.
- [5] A. Neumaier, „Rundungsfehleranalyse einiger Verfahren zur Summation endlicher Summen,” *ZAMM - Zeitschrift für Angewandte Mathematik und Mechanik*, 54(1): 39–51, 1974.
- [6] „OpenMP Application Programming Interface, Version 4.5.” *OpenMP Architecture Review Board*, Nov. 2015.
- [7] Материјали из предмета Мултипроцесорски системи, <http://mups.etf.rs/>, посећен 10.8.2026.
- [8] GCC Mirror on GitHub, "quadmath.h" Header file, <https://github.com/gcc-mirror/gcc/blob/master/libquadmath/quadmath.h>, посећен 16.8.2026.

СПИСАК СКРАЋЕНИЦА

IEEE - *Institute of Electrical and Electronics Engineers*

NaN - *Not-a-Number*

qNaN - *Quiet Not-a-Number*

sNaN - *Signaling Not-a-Number*

ULP - *Unit in the last place*

FORTTRAN - *Formula Translation*

ECM - *execution-cache-memory*

SIMD – *single-instruction-multiple-data*

RAM- *random-access-memory*

OpenMP - *Open Multi-Processing*

API - *Application Programming Interface*

GCC – *GNU Compiler Collection*

G++ – *GNU C++ Compiler*

СПИСАК СЛИКА

Слика 6.1.1. График зависности релативне грешке од броја нити и алгоритма, за апликацију сумирање низа, први тест пример	26
Слика 6.1.2. График зависности релативне грешке од броја нити и алгоритма, за апликацију сумирање низа, други тест пример.....	27
Слика 6.1.3. График зависности релативне грешке од броја нити и алгоритма, за апликацију сумирање низа, трећи тест пример.....	27
Слика 6.1.4. График зависности убрзања од броја нити и алгоритма, за апликацију сумирање низа за тест пример <i>mixed_random_10m</i>	28
Слика 6.2.1. График зависности глобалне релативне грешке од тест примера и алгоритма, за апликацију множење матрица	29
Слика 6.2.2. График зависности максималне апсолутне грешке елемента матрице од тест примера и алгоритма, за апликацију множење матрица	30
Слика 6.2.3. График зависности убрзања од броја нити и алгоритма, за апликацију множење матрица.....	31
Слика 6.3.1. График зависности глобалне релативне грешке силе од тест примера и алгоритма, за апликацију молекуларна динамика	33
Слика 6.3.2. График зависности максималне релативне грешке силе од тест примера и алгоритма, за апликацију молекуларна динамика	34
Слика 6.3.3. График зависности укупне суме свих сила на све честице од тест примера и алгоритма, за апликацију молекуларна динамика	34
Слика 6.3.4. График зависности убрзања од броја нити и алгоритма, за апликацију молекуларна динамика, за тест пример <i>md_3d_1000_500</i>	35

СПИСАК ТАБЕЛА

Табела 2.1.1.1. Параметри основних формата бројева у покретном зарезу	6
Табела 4.2.1. Спецификације сервера	18
Табела 6.1.1. Карактеристике тест примера за сумирање низа	25
Табела 6.2.1. Карактеристике тест примера за множење матрица	29
Табела 6.3.1. Карактеристике тест примера за апликацију молекуларна динамика	31